

UnitValues

Language Specification

Version 0.1.1

August 21, 2026

1 Introduction

1.1 Purpose

UnitValues is a set of domain-specific typed languages for numerical quantities. The set consists of two languages:

- `.ut` (unit types) for defining units from fundamental dimensions.
- `.uiv` (unit informed values) which import unit types and use dimensional construction to encode numerical quantities.

1.2 Scope

This document defines the syntax, semantics, and usage of all constructs in the `.ut` and `.uiv` domain-specific languages.

2 Language Rules

2.1 Global Section

The `[version]` section is required and defines the format version:

```
[version]
format: 0.1.1
```

2.2 Unit Frame

The `unit_frame` directive specifies which `.ut` file to import for unit definitions:

```
unit_frame: si_metric.ut
```

2.3 Sections

Sections are denoted by `[section.name]` and can be nested. Common section types include:

- `[standards]` — Defines standard values used across the file.
- `[material.meta]` — Metadata about a material (type, composition, notes).
- `[motor.magnetic]` — Magnetic properties (permeability, hysteresis).
- `[armature.core.slots]` — Armature slot properties (length, material).

2.4 Value Assignments

Values are assigned using the format:

```
key: value prefix(unit)
```

Examples:

- `density: 8930 (kg/m^3)`
- `temperature: 298.15 (K)`

2.5 Numeric Values

UnitValues supports real and complex numerical values. Complex values may be represented using the standard imaginary unit `j`:

```
real_value: 12.5 (V)
complex_value: 12.5+3.2j (V)
```

2.6 Unit Prefixes

SI prefixes can be applied to units:

- `(kg/mol)` — kilogram per mole.
- `M(A/m)` — mega-amp per meter.
- `m(m)` — millimeter (i.e., 1/1000 of a meter).
- `m(m^2)` — milli-square-meter (i.e., 1/1000 of a square meter).

Note:

Prefixes apply to the unit expression as a whole and do not distribute through its internal algebra. For example, `m(m^2)` represents 10^{-3} m^2 , not 10^{-6} m^2 .

2.7 Arrays and Tuples

Arrays are denoted with square brackets:

```
relative_permittivity: [1, 1]
```

Tuples (key-value pairs) use nested arrays:

```
temperature_conductivity: [
  [273.0, 401.0],
  [473.0, 389.0]
] (K), (k)
```

Note:

Nested arrays may represent column-wise data. When multiple unit expressions follow an array, each unit is assigned to the corresponding column. Thus, `(K), (k)` assigns kelvin to the first column and the derived unit `k` to the second column. The unit `k` is defined in the active unit frame.

2.8 Strings and Comments

Strings can be used as values:

```
my_spring: "Reduced Order Model | Tubular Ironless Linear Motor"
```

Comments begin with # and extend to the end of the line. Comments may appear on their own line or after a value:

```
# This is a comment
temperature: 298.15 (K) # Operating temperature
```

2.9 Unit Construction

Units are constructed from fundamental dimensions using the following operators:

Operation	Symbols	Description
Multiplication	$\ast$, x, $\cdot$, $\bullet$	Multiplies units together
Division	/, $\div$	Divides units
Power	$\wedge$	Raises a unit to a power

Examples:

```
kg*m^2*s^-3*A^-1
m/s^2
kg*m/s^2
```

3 Examples

3.1 .ut File Example

The following example demonstrates how to define custom units derived from fundamental dimensions in a .ut file:

```
# Example Units - Derived from Fundamental Dimensions (kg, m, s, A, etc.)
[version]
format: 0.1.1

[units]
# name: unit
p: kg*m^-1*s^-2          # Defines the unit for pressure (Pascal)
V: kg*m^2*s^-3*A^-1      # Defines the unit for voltage
```

Note:

The fundamental unit and prefix semantics (kg, m, s, A, etc.) & (u, m, k, M, etc.) is defined by the runtime environment.

3.2 .uiv File Example

The following example demonstrates how to use unit definitions from a .ut file and assign typed values in a .uiv file:

```
[version]
format: 0.1.1
unit_frame: units.ut

[model]
# name: value prefix(unit)
num_samples: 100                                # Implicitly dimensionless
sample_size: 10                                (∅)                # Explicitly dimensionless
output_energy: 1.0                               (kg*m^2*s^-2)          # Defines unit via construction
output_signal: 5.0                               (V)                   # Defined unit `V` for voltage
inlet_pressure: 101                             k(p)              # Defined unit with kilo prefix
```

4 Error Reference

4.1 Errors

1. **E001** — `ParserError`: Generic parser error that occurs when parsing fails. This is a catch-all error for unexpected parsing failures.
2. **E002** — `ParseListFailure`: Failed to parse a list or array structure. This occurs when a list definition (e.g., `[1, 2, 3]` or `(1, 2)`) is malformed or contains invalid elements that cannot be parsed correctly.
3. **E003** — `UnknownOperator`: Unsupported or unrecognized operator used in unit construction. Valid operators include `*`, `/`, `^`, etc.
4. **E004** — `UnknownPrefix`: Invalid or unrecognized prefix used in unit definitions. Valid prefixes include standard SI prefixes like `k`, `M`, `m`, `μ`, `n`, etc.
5. **E005** — `FailedCasting`: Failed to cast a value to a primitive type (e.g., converting a string to float or integer).
6. **E006** — `ColumnAttribute`: Nested array column index is out of range. Occurs when trying to access a column that doesn't exist in a nested array structure during attribute construction.
7. **E007** — `UnsupportedType`: Unsupported type encountered during unit construction. This occurs when a unit or dimensional cannot be converted to a valid unit type because the type is not supported.
8. **E008** — `UnbalancedDepth`: Unbalanced parentheses, brackets, or braces in the input. Occurs when opening and closing delimiters don't match properly during parsing.
9. **E009** — `InvalidSectionError`: Malformed section header in the configuration file. Section headers should follow the pattern `[SectionName]` and each section name must be a valid Python attribute name.
10. **E010** — `DuplicateSectionError`: Duplicate section found in the file. Each section header must be unique within a configuration file.

11. **E011** — `InvalidKeyError`: Malformed key-value pair. Occurs when a line in a section is not a valid Python attribute name.
12. **E012** — `UnitNotFoundError`: Unit doesn't exist in the current unit frame. Occurs when trying to use a unit that hasn't been defined in the available units list.

4.2 Warnings

1. **W001** — `BackCompatibilityWarning`: Missing `format` key in the version section. The file format version is required for proper parsing and backward compatibility with different file versions.
2. **W002** — `UnitFrameCompatibilityWarning`: Missing `unit_frame` key in the version section. The unit frame specification is needed for proper unit handling and compatibility with derived units.